A Mini Project Report On

A FACT-DIMENSION DATA-WAREHOUSE MODEL FOR HOSPITAL INSURANCE-CLAIMS DATA WAREHOUSE

Submitted by

Ms. Adapa Sai Sandhya **23ME1A5401**

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

IV B.Tech – I Semester (Mini Project)

Under the Esteemed Guidance of

Mrs. M.S.R. Pavani
Assistant Professor, Department of AI&DS



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

RAMACHANDRA COLLEGE OF ENGINEERING (AUTONOMOUS)

(Approved by AICTE, Permanantly Affiliated to JNTUK, Kakinada)
Accredited by NBA and NAAC (A+)
NH-16 Bypass, Vatluru (V), ELURU-534007, Eluru Dist., A.P.

2026–2027

RAMACHANDRA COLLEGE OF ENGINEERING (AUTONOMOUS)

(Approved by AICTE, Permanantly Affiliated to JNTUK, Kakinada) Accredited by NBA and NAAC (A+)

NH-16 Bypass, Vatluru (V), ELURU-534007, Eluru Dist., A.P.

2026–2027

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



BONAFIDE CERTIFICATE

This is to certify that this project report entitled "A FACT-DIMENSION DATA-WAREHOUSE MODEL FOR HOSPITAL INSURANCE-CLAIMS DATA WAREHOUSE" is being submitted by Adapa Sai Sandhya (23ME1A5401) in partial fulfillment of BACHELOR OF TECHNOLOGY in ARTIFICIAL INTELLIGENCE AND DATA SCIENCE.

This report is a bonafide work carried out under my guidance and supervision during the academic year 2026-2027 and it has been found worthy of acceptance as per the requirements of the university.

Mrs. M.S.R. Pavani	**Dr.P.Sudhakar**
PROJECT GUIDE	**Professor & HOD**

External Examiner

DECLARATION

I, Adapa Sai Sandhya (23ME1A5401), hereby declare that the project report titled "A FACT-DIMENSION DATA-WAREHOUSE MODEL FOR HOSPITAL INSURANCE-CLAIMS DATA WAREHOUSE" has been completed under the supervision of Mrs. M.S.R. Pavani, Assistant Professor**, Department of** Artificial Intelligence and Data Science. **This report is submitted in partial fulfillment of the requirements for the award of the degree of** BACHELOR OF TECHNOLOGY in ARTIFICIAL INTELLIGENCE AND DATA SCIENCE.

This is a record of work carried out by me, and the results presented in this project have not been reproduced or copied from any source. Furthermore, the results embodied in this project report have not been submitted to any other University or Institute for the award of any other degree or diploma.

Ms. Adapa Sai Sandhya **23ME1A5401**

Station: Eluru

Date:

ACKNOWLEDGEMENT

I take this opportunity to express my **sincere gratitude** to all those who have contributed to the successful completion of my project work. Their guidance, support, and encouragement have been invaluable throughout this journey.

I extend my **heartfelt thanks** to my guide, **Mrs. M.S.R. Pavani, Assistant Professor**, Department of **Artificial Intelligence and Data Science**, for the invaluable mentorship, insightful guidance, and unwavering support throughout this project. The expertise and constructive feedback have been instrumental in shaping this work.

I am deeply indebted to **Dr. P. Sudhakar, Professor & Head of the Department, Artificial Intelligence and Data Science**, for the continuous encouragement, guidance, and support in all aspects of this project.

My sincere recognition also goes to **Dr. M. Muralidhara Rao, Principal** for the valuable suggestions and constructive inputs during the preparation of this work.

I also wish to convey my sincere thanks to all the **Deans** for their constant encouragement and insightful suggestions.

I extend my profound gratitude to the **Management of Ramachandra College of Engineering, Eluru**, for their unwavering support, encouragement, and for providing the necessary facilities to carry out this project.

I would also like to express my deepest recognition to all the **faculty members and staff** of the Department of **Artificial Intelligence and Data Science** for their valuable advice and continuous motivation, which have played a crucial role in the successful completion of this project.

Finally, I extend my sincere gratitude to my parents, friends and everyone who has directly or indirectly contributed to the successful completion of this project work.

Ms. Adapa Sai Sandhya **23ME1A5401**

TABLE OF CONTENTS

SL.NO	TITLE	PAGE NO
	BONAFIDE CERTIFICATE	**i**
	DECLARATION	**ii**
	ACKNOWLEDGEMENT	**iii**
	ABSTRACT	**vi**
	LIST OF FIGURES	**vii**
1.	**CHAPTER -1**	**1 - 3**
	INTRODUCTION	
	1.1 Introduction to Hospital Insurance-Claims Data Warehousing	1
	1.2 Problem Statement	2
	1.3 Objectives of the Project	2
	1.4 Scope of the Project	3
	1.5 Organization of the Report	3
2.	**CHAPTER -2**	**4 - 7**
	SYSTEM ANALYSIS	
	2.1 Existing Methods and Literature Review	4
	2.2 Limitations of Existing System	5
	2.3 Proposed System	6
	2.4 Advantages of Proposed System	6
3.	**CHAPTER -3**	**8 - 9**
	SYSTEM REQUIREMENT SPECIFICATION	
	3.1 Hardware Requirements	8
	3.2 Software Requirements	8
	3.3 Functional Requirements	8
	3.4 Non-Functional Requirements	9
4.	**CHAPTER -4**	**10 - 15**
	SYSTEM DESIGN	
	4.1 Architecture Diagram	10
	4.2 Data Flow Diagram (DFD)	11
	4.3 UML Diagrams	12
	4.4 Database / Dataset Design (Fact-Dimension Model)	13
5.	**CHAPTER -5**	**16 - 18**
	MODULE DESCRIPTION	
	5.1 Source Data Simulation and Extraction	16
	5.2 Data Quality and Staging	16
	5.3 Dimension Modelling (SCD Type-1 and Type-2)	16
	5.4 Fact Loading and Snapshot Construction	17
	5.5 OLAP Analytics, Testing and Reporting	17
6.	**CHAPTER -6**	**19 - 23**
	SYSTEM IMPLEMENTATION	

	6.1 Source Data Simulation and Extraction Implementation	19
	6.2 Data Quality and Staging Implementation	19
	6.3 Dimension Modelling and SCD Implementation	21
	6.4 Fact Loading, Snapshot and Final Integration Implementation	22
7.	**CHAPTER -7**	**24 - 28**
	TESTING AND OUTPUT SCREENS	
	7.1 Test Cases and Test Results	24
	7.2 Output Screen 1 – ETL Execution Console	25
	7.3 Output Screen 2 – Claims Analytics Dashboard	26
	7.4 Performance Results and Discussion	27
8.	**CONCLUSION AND FUTURE ENHANCEMENTS**	**29 - 30**
	8.1 Conclusion	29
	8.2 Future Enhancements	29
	REFERENCES	**31**
	APPENDIX	**32**

RAMACHANDRA COLLEGE OF ENGINEERING
AUTONOMOUS

Approved by AICTE, New Delhi
Permanently Affiliated to JNTUK
Recognized by UGC 2(f) & 12(B)
Accreditations
NAAC A+
NBA (EEE, Civil, ME, ECE & CSE)
ISO 9001 : 2015 Certified

ABSTRACT

TITLE: A FACT-DIMENSION DATA-WAREHOUSE MODEL FOR HOSPITAL INSURANCE-CLAIMS DATA WAREHOUSE

Hospitals submit thousands of insurance claims every month, and every claim passes through admission, billing, submission, adjudication and payment. The data sit in transactional billing systems built for recording events, not for analysis, so questions about denial patterns, settlement delays and revenue leakage need slow multi-table joins over duplicated and inconsistent records. Existing healthcare warehouse studies concentrate on clinical data or national-scale platforms, and few describe a compact dimensional model for hospital claims.

This project designed and implemented a fact-dimension (star schema) data warehouse for hospital insurance claims. The model contained a claim-line transaction fact table, a monthly snapshot fact table and nine conformed dimensions, including a Type-2 slowly changing patient dimension. A Python and SQLite pipeline loaded a synthetic but realistic source system of 100,000 claims and 571,145 claim lines from eight hospitals and six insurers over three years, with deliberately injected data defects.

The work was organised into five modules: source simulation and incremental extraction, data-quality gating with six rules, dimension modelling with Type-1 and Type-2 history, fact loading with surrogate-key and point-in-time lookup, and OLAP analytics with automated testing. Late-arriving adjudications were applied by upsert, rejected records were logged, and every batch was audited so that loads could be replayed safely.

The warehouse stored 568,643 claim lines after rejecting 2,502 invalid lines, and all 13 automated test cases passed. Six OLAP queries returned results identical to those from a normalized copy; the star schema was faster on 4 of 6 queries (up to 5.2 times) and the monthly snapshot answered the trend query in 0.3 ms against 865 ms. The model can support claims analysis in hospital finance and insurance-desk teams.

Keywords: data warehouse, star schema, fact table, dimension table, insurance claims, ETL, OLAP, slowly changing dimension

LIST OF FIGURES

SI NO	IMAGE NAME	PAGE NO
1	System Architecture	10
2	Data Flow Diagram – Level 0	11
3	Data Flow Diagram – Level 1	11
4	Use Case Diagram	12
5	Sequence Diagram	12
6	Star Schema (Fact-Dimension Model)	14
7	Module Interaction Flow	18
8	ETL Batch Flowchart	21
9	ETL Load Profile and Data-Quality Outcomes	23
10	Output Screen 1 – ETL Execution Console	25
11	Output Screen 2 – Claims Analytics Dashboard	26
12	Query Response-Time Comparison	28

CHAPTER - 1

INTRODUCTION

1.1 Introduction to Hospital Insurance-Claims Data Warehousing

Health insurance, whether private, employer-sponsored or government-funded, pays for a growing share of hospital treatment. For a hospital this turns every admission into two parallel processes: delivering care and recovering the cost of that care from an insurer or a third-party administrator. A single claim passes through registration, diagnosis coding, itemised billing, submission, adjudication, partial or full payment and, in many cases, rejection and resubmission. The money tied up in this cycle decides how quickly a hospital can pay its staff, buy supplies and invest in equipment.

The data produced by this cycle are large and detailed. A single inpatient claim can carry tens of service lines such as bed charges, investigations, procedures and pharmacy items, each with billed, allowed, denied and paid amounts. These records live in transactional (OLTP) billing systems that are normalized for fast and consistent data entry. That design is excellent for recording a claim but poor for answering management questions, such as which insurer denies the most claims for a given diagnosis, how long each insurer takes to pay, or how the settlement ratio of a hospital changes from month to month. Such questions need many joins and aggregations over millions of rows, and the answers differ between departments because each team cleans the data in its own way.

A data warehouse addresses this gap. Inmon defined a data warehouse as a subject-oriented, integrated, time-variant and non-volatile collection of data that supports management decisions [2]. Kimball and Ross proposed dimensional modelling, in which numeric measurements of a business process are stored in a fact table surrounded by descriptive dimension tables, forming a star schema that is easy for analysts to understand and fast to query [1]. Chaudhuri and Dayal describe how this structure supports online analytical processing (OLAP) operations such as roll-up, drill-down, slice and dice [3].

Recent reviews show that healthcare organisations increasingly adopt warehouses, but most published work targets clinical or research data. A scoping review of 137 clinical data warehousing articles found that data integration dominated the literature while data quality, governance and sustainability received little attention [4], and another review of 27 studies reported that general-purpose warehouses mostly rely on star schemas with OLAP, whereas specialised warehouses depend on data mining and AI but struggle with scalability and

interoperability [5]. Detailed designs for hospital-side insurance-claims warehouses, including their data-quality handling and history tracking, are rarely documented.

This project follows the four-step dimensional design process of Kimball and Ross [1]: choose the business process (hospital insurance-claim processing), declare the grain (one claim service line), choose the dimensions (date, patient, provider, hospital, diagnosis, procedure, insurer, plan and claim status) and identify the facts (quantity, billed, allowed, denied, co-pay and paid amounts together with processing time). The design was implemented as a working warehouse, loaded by an incremental ETL pipeline, validated by automated tests and evaluated against a normalized copy of the same data.

1.2 Problem Statement

Hospital finance and claims teams must answer recurring questions about denials, delays and settlement, but the operational billing database stores claims across many normalized tables, contains duplicated and invalid records, and overwrites history when a patient changes plan or city. Analysts therefore rely on slow joins, hand-cleaned spreadsheets and inconsistent definitions, which leads to unreliable denial rates and late action on revenue leakage. Published warehouse studies mostly address clinical data, and those that handle claims operate at national scale on heavy platforms. There is a need for a compact, well-documented fact-dimension model for hospital insurance claims that cleans data on load, preserves history, supports late-arriving status changes and answers analytical queries quickly.

1.3 Objectives of the Project

- To design a fact-dimension (star schema) data-warehouse model for hospital insurance claims, with a claim-line transaction fact table, a monthly snapshot fact table and nine conformed dimensions.
- To develop an incremental ETL pipeline in Python that extracts changed records, applies six data-quality rules, resolves surrogate keys and loads facts by upsert.
- To preserve patient history with a Type-2 slowly changing dimension and a point-in-time lookup, and to apply late-arriving claim adjudications without duplicating rows.
- To evaluate the warehouse using 13 automated test cases and six OLAP queries, and to compare query results and response times with a normalized (3NF) copy of the same data.
- To present the results through an ETL execution log, an analytics dashboard and a response-time comparison.

- To keep the solution reproducible by generating a synthetic source system with injected defects, so that every reported number can be regenerated.

1.4 Scope of the Project

The project covers the complete path from a hospital claims source system to an analytical warehouse: simulation of the source tables (claims, claim lines, patients, plans, insurers, hospitals, doctors, diagnosis and procedure references), incremental extraction, data-quality control, dimensional modelling, fact loading, aggregate snapshot construction, OLAP querying and reporting. The intended users are hospital finance managers, claims analysts, insurer-liaison staff and the data engineer who operates the load. The data cover three calendar years (2023 to 2025), eight hospitals, six insurers, eighteen plans, 30 ICD-10 diagnosis codes (plus an Unknown member) and 24 service categories.

The project does not use real patient or insurer records, because such data are protected; a synthetic source system with realistic distributions and injected defects was generated instead, and all reported figures describe that data. The warehouse runs on SQLite on a single machine and is not a distributed or real-time system. Fraud scoring, predictive modelling, clinical data, user authentication and a web front end are outside the scope; the dashboard is produced as static charts from warehouse queries.

1.5 Organization of the Report

Chapter 2 reviews existing methods and literature, lists their limitations and presents the proposed system and its advantages.

Chapter 3 specifies the hardware, software, functional and non-functional requirements.

Chapter 4 presents the system design: architecture, data flow diagrams, UML diagrams and the database (star schema) design.

Chapter 5 describes the five modules of the system and how they interact.

Chapter 6 explains the implementation of each module with algorithms and key code.

Chapter 7 reports the test cases, output screens and performance results.

Chapter 8 concludes the work and lists future enhancements. References and appendices follow.

CHAPTER - 2

SYSTEM ANALYSIS

2.1 Existing Methods and Literature Review

The literature relevant to this project falls into four groups: reviews of warehouse practice in healthcare, dimensional warehouse designs, standardisation of claims data, and analytics on claims data.

Reviews of healthcare warehousing. Wang et al. screened clinical data warehouse articles published between 2011 and 2021 and analysed 137 of them; most were case studies, data integration was the most common focus, and data quality, governance, staff training and financial sustainability were rarely studied [4]. Lyu et al. followed a PRISMA-guided scoping review of 27 studies from 2014 to 2024 and found that 17 general-purpose warehouses mainly integrated EHR data using star schemas and OLAP, while 10 specialised warehouses relied on data mining and AI but suffered scalability and interoperability problems; they suggested hybrid architectures and standard frameworks [5].

Dimensional warehouse designs. Turcan and Peker applied Kimball's bottom-up methodology to build a fact-constellation schema with seven fact tables for COVID-19 data, covering demographics, tests, vaccines, quarantine, venues, drugs and symptoms, and demonstrated seven sample queries with acceptable response times [6]. Rinner et al. used the Kimball lifecycle to build a clinical warehouse for Austrian health claims in the OMOP common data model queried through i2b2, using synthetic data with about 4.3 million patients, 460,000 hospital stays and 10.3 million medications over three years, and produced reusable Austrian dimensions such as insurers, regions and care sites [10]. Su outlined key technologies for insurance claims, including warehouses, OLAP and ETL, within a CRISP-DM data-mining process on a 29-variable claims dataset, without reporting quantitative results [11].

Standardising claims data. Kim et al. converted the South Korean national insurance claims database of about 10 billion claims from 56.6 million patients into the OMOP model, built a privacy-preserving distributed analysis environment, and validated it by reproducing diabetes case counts and a COVID-19 prediction model [7]. Finster et al. built an ETL pipeline that mapped two German Health Data Lab claims formats to OMOP, reaching field coverage of 92.7 percent and 86.2 percent for the two formats, although concept mapping was weak where test data were invalid or mocked [8].

Analytics on claims. Cherkaoui et al. benchmarked six supervised classifiers and two categorical-feature encodings for health insurance fraud detection on record-level claims data

with expert-rule labels, raising detection accuracy 4.2 times and reaching 84 percent recall at a 20 percent positive rate [9]. Such models presuppose clean, integrated claims data, which is precisely what a warehouse is meant to provide.

2.2 Limitations of Existing System

The reviewed work shows clear progress but leaves the following limitations, which motivate the proposed system. The table summarises each study.

- Most designs target clinical or national research data; hospital-side claims warehouses at claim-line grain are not documented in detail [4], [5], [10].
- Data-quality handling, duplicate removal and reject logging are rarely described, although they decide the trustworthiness of denial and payment metrics [4], [8].
- History tracking (slowly changing dimensions) and late-arriving status changes are not reported for claims, so attributes such as a patient's plan or city lose their time context [6], [10].
- Performance claims are qualitative ("proper response times") or absent, with no comparison against the operational schema [6], [11].
- Standardising models such as OMOP or distributed national platforms are heavy for a single hospital and are not star schemas that analysts can query with plain SQL [7], [8], [10].
- Fraud-detection models assume prepared data and do not describe how that data are produced and kept consistent [9].

Ref	Method / Technique	Dataset	Result	Limitation
[4]	Scoping review of clinical data warehousing (PubMed, 2011 to 2021)	137 articles	Integration most studied; quality and governance neglected	Descriptive; no design or benchmark
[5]	PRISMA scoping review of warehouses in clinical settings	27 studies (2014 to 2024)	General: star schema and OLAP; specialised: mining and AI	Scalability and interoperability gaps
[6]	Kimball bottom-up fact constellation, 7 fact tables	COVID-19 warehouse structure	7 sample queries returned in proper time	No timings, ETL or history handling
[7]	HIRA claims converted to OMOP, distributed analysis	10 billion claims, 56.6 M patients	Case counts matched; model replicated	Heavy infrastructure; not a star schema
[8]	ETL of German claims formats to OMOP	Health Data Lab claims	Field coverage 92.7% and 86.2%	Weak concept mapping on mock data
[9]	Benchmark of 6 classifiers, 2 encodings for fraud	Record-level insurance claims	Accuracy x4.2; recall 84% at 20% positive rate	Needs prepared data; rule-based labels
[10]	Kimball lifecycle, OMOP and i2b2 for claims	Synthetic Austrian claims, 4.3 M patients	Reusable dimensions, i2b2 ontology	Insurer side; no history or timings
[11]	CRISP-DM with warehouse, OLAP and ETL	29-variable claims dataset	No quantitative results	Conceptual only

2.3 Proposed System

The proposed system, named the Hospital Insurance-Claims Data Warehouse, converts a hospital's normalized billing data into a star schema designed for claims analysis. The grain of the main fact table is one claim service line. Each fact row carries three role-playing date keys (service, submission and adjudication), eight dimension keys, a degenerate claim identifier, and additive measures for quantity, billed, allowed, denied, co-pay and paid amounts. A second, periodic snapshot fact table stores monthly totals by insurer and hospital so that trend questions are answered without scanning claim lines.

To overcome the limitations in Section 2.2, the pipeline is incremental and defensive. Each batch reads only rows changed since the stored watermark, removes duplicate claim headers, and applies five line-level rules and one mapping rule before any data reach the warehouse; every violation is written to a reject table with its rule identifier. Reference attributes are loaded as Type-1 dimensions, while the patient dimension is Type-2: when a patient's city or plan changes, the old row is closed and a new version is opened, and each claim is linked to the version valid on its admission date. Claims that were pending in one batch and adjudicated later are updated in place by an upsert on the natural key (claim identifier and line number).

The warehouse is validated by 13 automated test cases covering reconciliation, referential integrity, deduplication, rejection, unknown-member mapping, history, calendar continuity, snapshot consistency, query equivalence, late updates and idempotent reloading. Analytical value is shown by six OLAP queries and a dashboard, and by a response-time comparison against a normalized copy of the same data.

End to end, source rows flow from the billing system through extraction, quality gating, dimension building and fact loading into the star schema, and then to analysts through SQL and charts, with an audit record written for every batch.

2.4 Advantages of Proposed System

- **Trustworthy metrics:** six data-quality rules and a reject log keep invalid records out of the fact table and make every exclusion traceable.
- **History preserved:** Type-2 patient versions and point-in-time lookup keep claims linked to the plan and city that applied when care was given.
- **Always current:** incremental loading with an upsert applies late adjudications and can be replayed safely without creating duplicate rows.
- **Fast analysis:** the star schema needs few joins, and the monthly snapshot answered the trend query in 0.3 ms.

- **Low cost and portable:** only open-source tools (Python, SQLite, pandas, matplotlib) are required, and the SQL is standard enough to move to MySQL or PostgreSQL.
- **Extendable:** conformed dimensions allow new fact tables, for example pre-authorisation or payment events, to be added later.

CHAPTER - 3

SYSTEM REQUIREMENT SPECIFICATION

3.1 Hardware Requirements

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or above
RAM	8 GB minimum (16 GB recommended for the full load)
Storage	256 GB SSD (the three databases together use about 0.3 GB)
GPU (if used)	Not required
Other	None; a standard display and keyboard

3.2 Software Requirements

Software	Specification
Operating System	Windows 10/11 or Ubuntu 22.04 LTS
Programming Language	Python 3.10 or above (developed with Python 3.13)
Libraries / Frameworks	pandas 3.0, NumPy 2.5, matplotlib 3.11, sqlite3 module [12], [13], [14]
Database	SQLite 3.45 (portable to MySQL or PostgreSQL)
IDE / Tools	VS Code, Jupyter Notebook, DB Browser for SQLite
Browser	Chrome / Edge (for viewing exported reports)

3.3 Functional Requirements

FR1 – The system shall extract only the claim, claim-line and patient records that changed since the last successful batch (watermark).

FR2 – The system shall remove duplicate claim headers and log each removal.

FR3 – The system shall reject claim lines with negative billed amount, paid amount above allowed amount, service date outside the warehouse calendar, or no parent claim, and log each rejection with its rule.

FR4 – The system shall map missing or invalid diagnosis codes to an Unknown member instead of dropping the claim.

FR5 – The system shall maintain Type-1 dimensions for hospital, provider, diagnosis, procedure, insurer, plan and claim status.

FR6 – The system shall maintain the patient dimension as Type-2, closing the old version and opening a new one when city, state or plan changes.

FR7 – The system shall load claim lines into the fact table using surrogate keys and the patient version valid on the admission date, and shall update existing lines when adjudication results arrive late.

FR8 – The system shall rebuild a monthly snapshot fact table by insurer and hospital after every batch.

FR9 – The system shall record every batch in an audit table with counts, watermark and duration.

FR10 – The system shall answer roll-up, drill-down, slice and dice queries and produce a claims dashboard.

3.4 Non-Functional Requirements

The quality requirements below are stated with target values and the values observed in the tests.

- **Performance** – analytical queries on about 569 thousand fact rows should finish in under one second; measured median times were 96 to 589 ms, and an incremental batch of about 220 thousand lines loaded in 13 s.
- **Security** – patient names and policy numbers are not loaded into the warehouse; analysts see only surrogate keys, demographics and codes. Database files can be protected by operating-system permissions.
- **Usability** – the star schema uses business names (insurer, hospital, diagnosis) so that analysts can write simple SQL; results are also presented as a dashboard.
- **Reliability and Availability** – invalid data never stop a batch; they are rejected and logged. Loads are idempotent, so a failed or repeated batch can be replayed without duplicates.
- **Scalability** – the incremental design processes only changed rows, and the fact table can be partitioned or moved to a server database as volume grows.
- **Maintainability and Portability** – each module is a separate Python function or file with a single responsibility, and the pipeline runs on Windows and Linux without changes.

CHAPTER - 4

SYSTEM DESIGN

4.1 Architecture Diagram

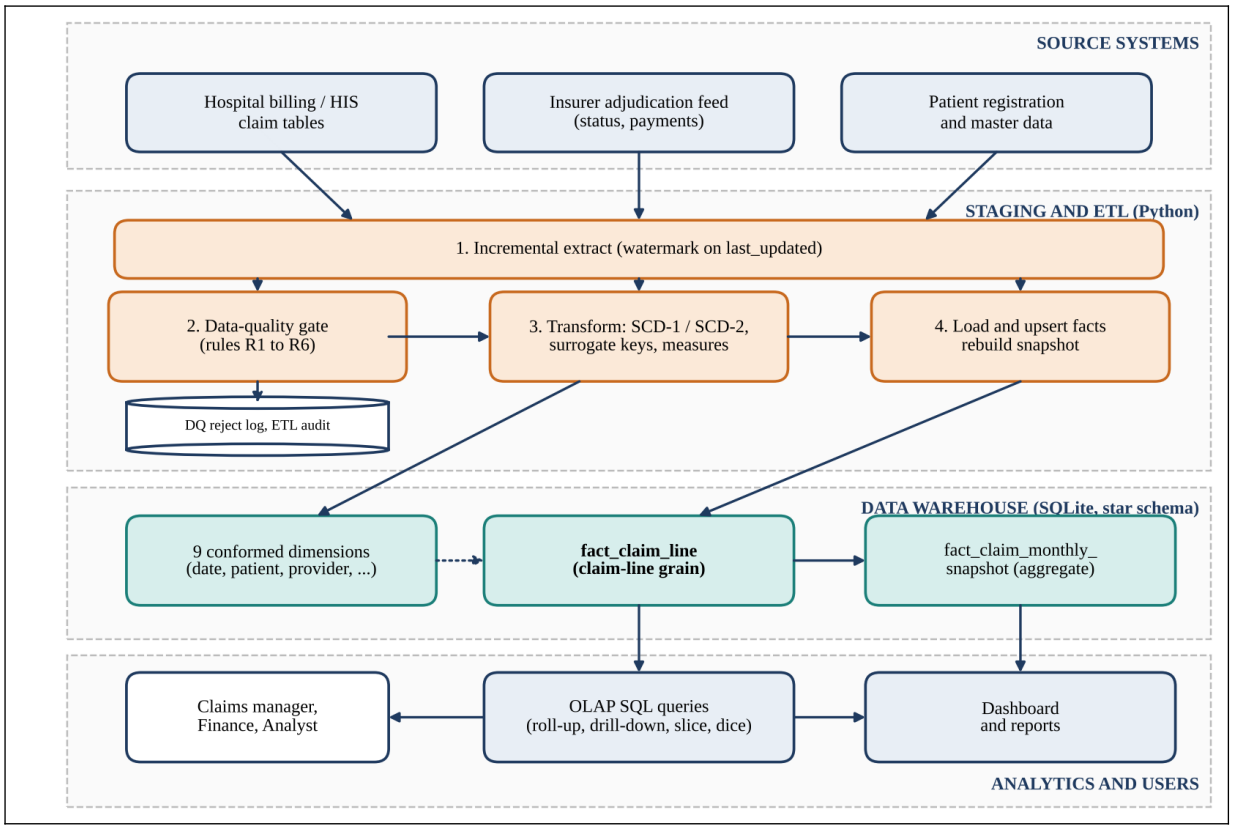


Fig. 1: System Architecture

Fig. 1 shows the layered architecture. The **source layer** contains the hospital billing / HIS claim tables, the insurer adjudication feed (claim status, denial reason and payment amounts) and the patient registration master. In this project the three sources are simulated in one normalized SQLite database. The **staging and ETL layer** is a Python pipeline of four steps: incremental extract, data-quality gate, transform (dimension maintenance, surrogate keys and measures) and load. Rejected records and batch statistics are written to a reject log and an audit table.

The **warehouse layer** holds nine conformed dimensions, the transaction fact table fact_claim_line at claim-line grain and the periodic snapshot fact_claim_monthly_snapshot. The **analytics layer** consists of OLAP SQL queries, which support roll-up, drill-down, slice and dice, and a dashboard that is rendered from the query results for claims managers, finance staff and analysts.

Data move strictly downstream. Only rows changed since the watermark enter the pipeline, valid rows reach the dimensions and the fact table, the snapshot is rebuilt from the fact table,

and users read only from the warehouse, so analytical queries never load the operational billing system.

4.2 Data Flow Diagram (DFD)

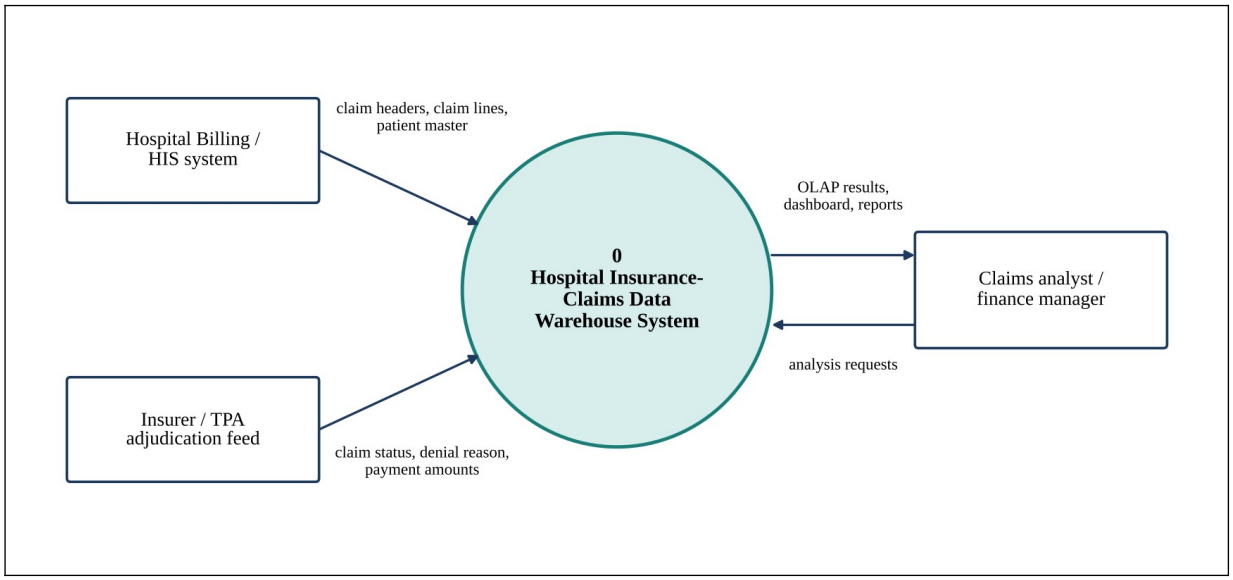


Fig. 2: Data Flow Diagram – Level 0

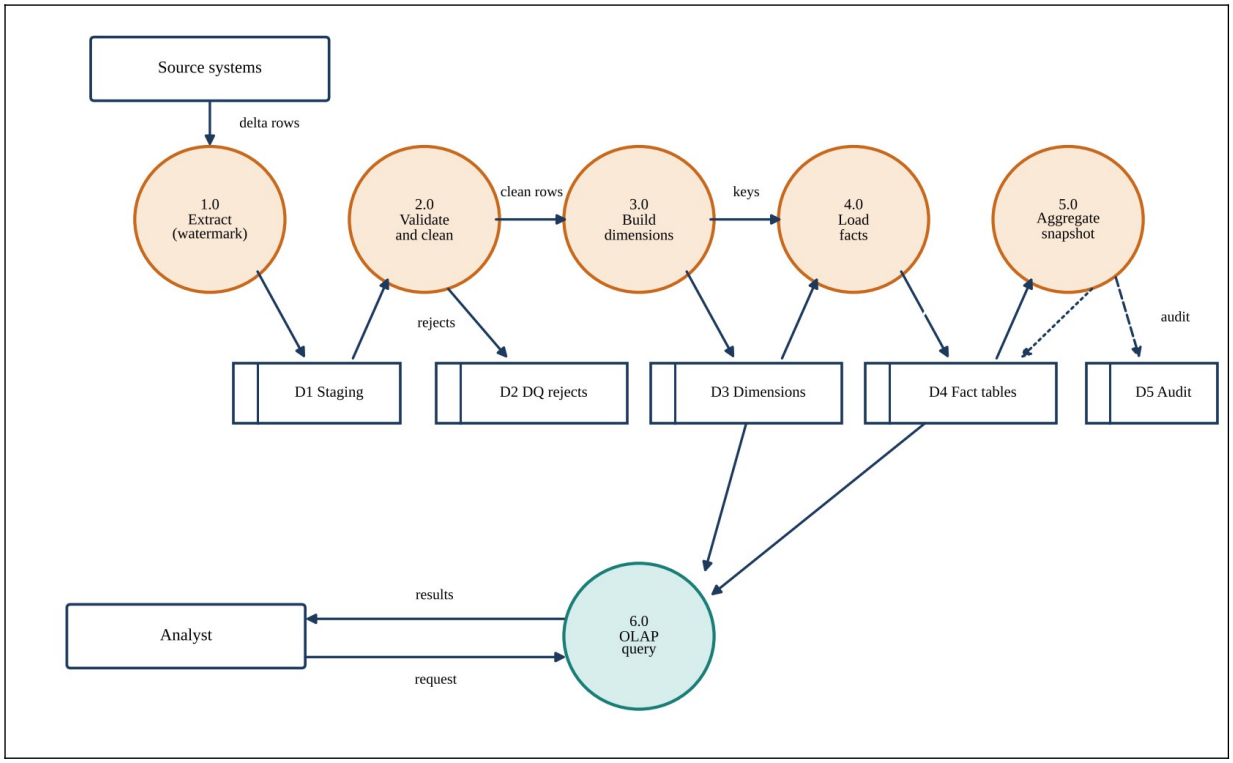


Fig. 3: Data Flow Diagram – Level 1

The context diagram (Fig. 2) shows the system as a single process that receives claim headers, claim lines and patient master data from the hospital billing system, and claim status, denial reasons and payment amounts from the insurer or TPA feed, and returns OLAP results, dashboards and reports to the claims analyst or finance manager, who sends analysis requests.

Level 1 (Fig. 3) expands the system into six processes: 1.0 extract (watermark), 2.0 validate and clean, 3.0 build dimensions, 4.0 load facts, 5.0 aggregate snapshot and 6.0 OLAP query. The data stores are D1 staging, D2 data-quality reject log, D3 dimensions, D4 fact tables and D5 ETL audit. Process 2.0 writes every rejected record to D2, process 4.0 writes the batch summary to D5, and process 6.0 reads D3 and D4 to answer requests.

4.3 UML Diagrams

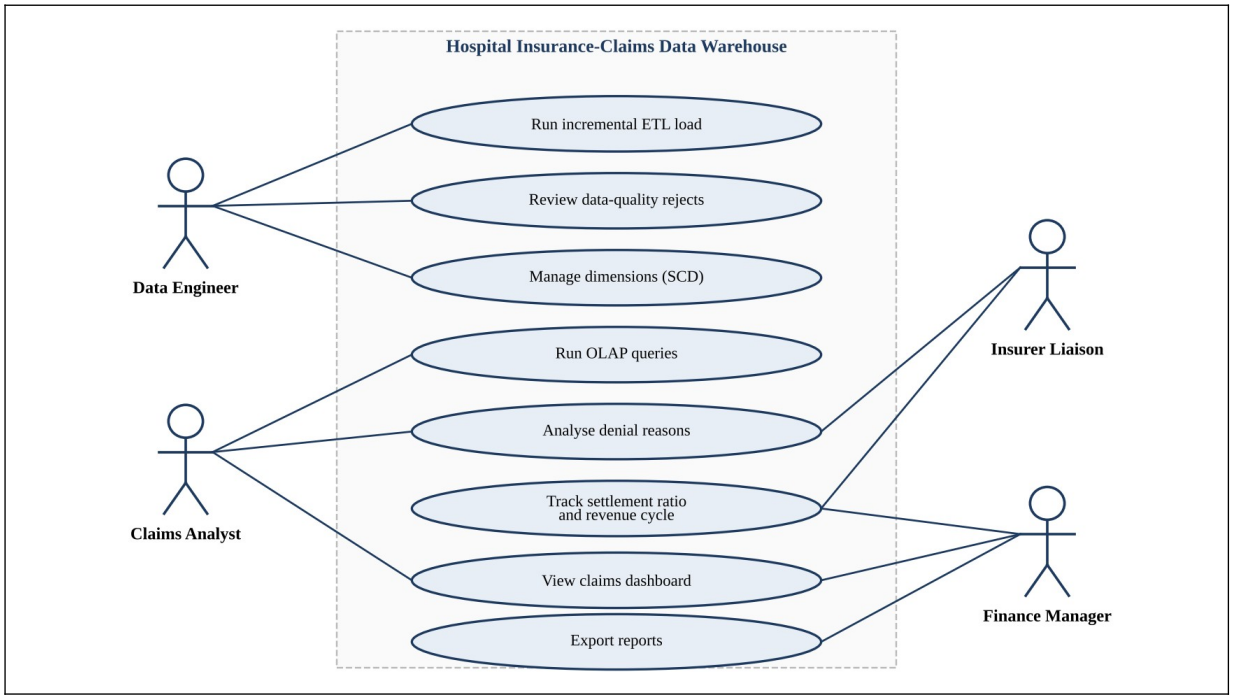


Fig. 4: Use Case Diagram

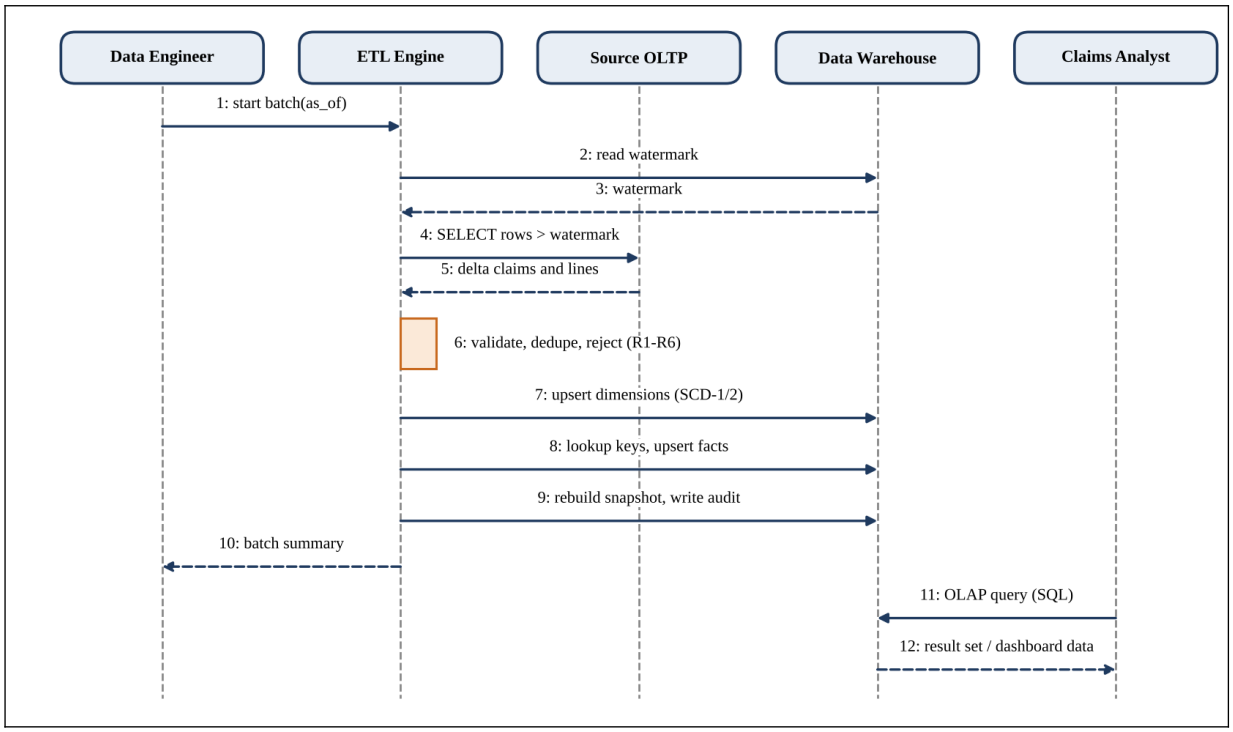


Fig. 5: Sequence Diagram – ETL Batch and OLAP Query

The use case diagram (Fig. 4) has four actors. The data engineer runs the incremental ETL load, reviews data-quality rejects and manages dimensions. The claims analyst runs OLAP queries, analyses denial reasons and views the dashboard. The insurer liaison analyses denial reasons and tracks settlement and the revenue cycle, and the finance manager tracks settlement, views the dashboard and exports reports.

The sequence diagram (Fig. 5) shows one batch: the engineer starts the batch with a business date; the ETL engine reads the watermark from the warehouse, selects rows newer than the watermark from the source, validates and deduplicates them, upserts dimensions and facts, rebuilds the snapshot and writes the audit record, and returns a batch summary. Afterwards the analyst sends an OLAP query to the warehouse and receives a result set.

4.4 Database / Dataset Design (Fact-Dimension Model)

The warehouse follows the dimensional design steps of Kimball and Ross [1]. The business process is hospital insurance-claim processing. The **grain** of fact_claim_line is one claim service line (one claim_id and line_no). The nine dimensions are conformed, so the transaction fact and the snapshot fact share dim_insurer, dim_hospital and dim_date. Three role-playing date keys (service, submission and adjudication) reference the single dim_date table, and the value 0 means that a claim has not yet been adjudicated.

All measures are additive (quantity, billed, allowed, denied, co-pay and paid amounts) except processing_days, which is a non-additive lag that is averaged. To count claims without COUNT(DISTINCT), each line carries claim_weight = 1 / (number of lines in the claim), so that the sum of claim_weight over any set of lines equals the number of claims. The claim status, denial reason, claim type and admission type are low-cardinality flags and are combined in one junk dimension of 72 members. dim_patient is Type-2; the other dimensions are Type-1. dim_diagnosis contains the Unknown member (key -1). Fig. 6 shows the schema and the table below lists the fields.

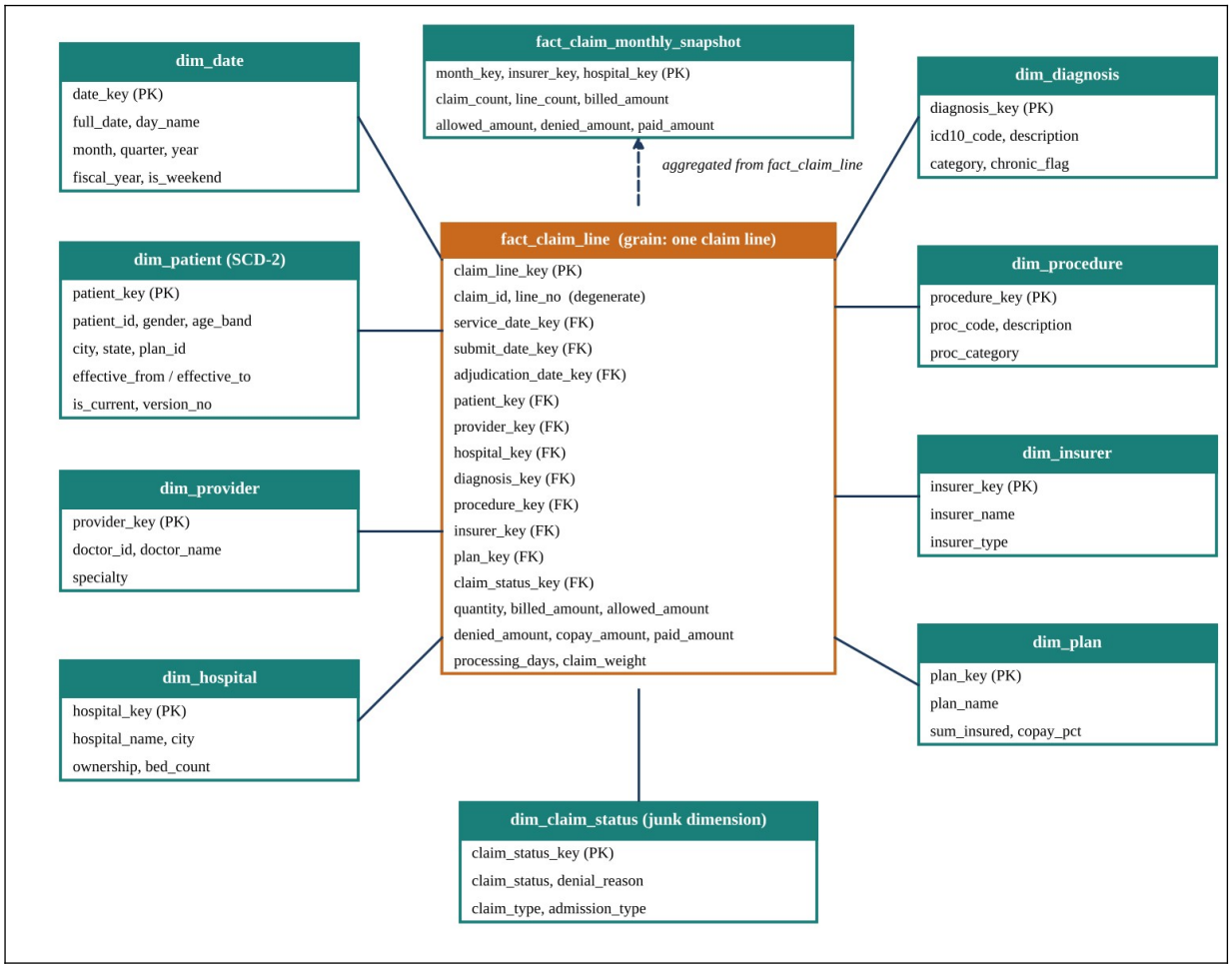


Fig. 6: Star Schema (Fact-Dimension Model) of the Claims Warehouse

Table / File	Field	Type	Description
fact_claim_line	claim_line_key	INTEGER PK	Surrogate key of the fact row
	claim_id, line_no	TEXT, INTEGER	Degenerate dimension; natural key (unique)
	service / submit / adjudication _date_key	INTEGER FK	Role-playing dates; 0 = not yet adjudicated
	patient_key, provider_key, hospital_key, diagnosis_key, procedure_key, insurer_key, plan_key, claim_status_key	INTEGER FK	Links to the dimensions
	quantity	INTEGER	Units of service
	billed_amount, allowed_amount	REAL	Additive measures (INR)
	denied_amount, copay_amount, paid_amount	REAL	Additive; paid = allowed − denied − co-pay
	processing_days, claim_weight	INTEGER, REAL	Submit-to-adjudication lag; 1 / lines per claim
fact_claim_monthly_snapshot	month_key, insurer_key, hospital_key	INTEGER PK	Grain: month x insurer x hospital
	claim_count, line_count, billed, allowed, denied, paid	REAL	Aggregates of the transaction fact
dim_date	date_key, full_date, day_name, month, quarter, year, fiscal_year, is_weekend	INTEGER PK ...	Calendar 2022 to 2026 (1,827 rows), fiscal year April to March
dim_patient (SCD-2)	patient_key, patient_id, gender, birth_date, age_band	INTEGER PK, TEXT	Surrogate and natural key; descriptive attributes
	city, state, plan_id, effective_from, effective_to, is_current, version_no	TEXT, INTEGER	Tracked attributes and version control
dim_provider	provider_key, doctor_id, doctor_name, specialty	INTEGER PK, TEXT	Treating doctor
dim_hospital	hospital_key, hospital_id, hospital_name, city, ownership, bed_count	INTEGER PK, TEXT	Facility

Table / File	Field	Type	Description
dim_diagnosis	diagnosis_key, icd10_code, description, category, chronic_flag	INTEGER PK, TEXT	Primary diagnosis; key -1 = Unknown
dim_procedure	procedure_key, proc_code, description, proc_category	INTEGER PK, TEXT	Service category of the line
dim_insurer	insurer_key, insurer_id, insurer_name, insurer_type	INTEGER PK, TEXT	Private, TPA or Government
dim_plan	plan_key, plan_id, plan_name, sum_insured, copay_pct	INTEGER PK, TEXT	Insurance plan terms
dim_claim_status	claim_status_key, claim_status, denial_reason, claim_type, admission_type	INTEGER PK, TEXT	Junk dimension, 72 combinations
dq_reject, etl_audit	batch_id, entity, record_id, rule_id, ... / counts, watermark, seconds	TEXT, INTEGER	Reject log and batch audit

CHAPTER - 5

MODULE DESCRIPTION

The system is divided into five modules. For each module the purpose, input, processing and output are described below.

5.1 Source Data Simulation and Extraction

Purpose: to provide a realistic hospital billing source system and to extract only new or changed records in each batch.

Input: reference data (hospitals, insurers, plans, doctors, ICD-10 diagnoses, service categories) and parameters such as the number of patients and claims. **Process:** the generator creates 25,000 patients and 100,000 claims whose diagnosis mix, seasonality, hospital tariffs, insurer denial tendencies and adjudication delays differ by design. It produces itemised claim lines (about 5.7 per claim) and injects defects: 400 missing and 298 invalid diagnoses, 249 duplicate claim headers, 834 negative amounts, 1,112 lines with paid above allowed and 556 invalid service dates. Batch 1 (as of 31 Dec 2024) loads claims submitted until then, with later-adjudicated claims shown as Pending; batch 2 (as of 31 Dec 2025) adds 2025 claims, updates the pending ones and applies 2,391 patient plan or city changes. The extractor reads rows whose last_updated is newer than the stored watermark.

Output: delta tables of claims, claim lines and patients, and the new watermark value.

5.2 Data Quality and Staging

Purpose: to stop invalid or duplicate data from reaching the warehouse while keeping a complete record of what was excluded and why.

Input: the delta tables from module 5.1. **Process:** rule R1 removes duplicate claim headers (first occurrence kept); R2 rejects negative billed amounts; R3 rejects lines with paid amount above allowed amount; R4 rejects service dates outside the warehouse calendar; R5 rejects lines with no parent claim; R6 maps a missing or unknown ICD-10 code to the Unknown diagnosis member and keeps the claim. Each event is written to dq_reject with its batch, entity, record identifier, rule and action.

Output: clean claim and claim-line sets, and the reject log used for data-quality reporting.

5.3 Dimension Modelling (SCD Type-1 and Type-2)

Purpose: to build conformed dimensions with correct history.

Input: clean reference tables and the patient master. **Process:** the calendar dimension is generated once. Hospital, provider, diagnosis, procedure, insurer and plan are Type-1: new natural keys are inserted and changed attributes overwritten. The claim-status junk dimension receives any new combination of status, denial reason, claim type and admission type. The patient dimension is Type-2: patients are compared with their current version on city, state and plan; if one differs, the current row receives an expiry date and is_current = 0, and a new row with the next version number, effective from the profile update date, is inserted. The derived age band is refreshed on all versions.

Output: dimension tables with surrogate keys, ready for key lookup.

5.4 Fact Loading and Snapshot Construction

Purpose: to convert clean claim lines into fact rows and to maintain the aggregate snapshot.

Input: clean claims and lines, dimension keys. **Process:** each line is joined to its claim and mapped to surrogate keys; the patient version is chosen so that the admission date lies between effective_from and effective_to (point-in-time lookup). The date keys, processing_days and claim_weight are computed. Rows are written by an upsert on (claim_id, line_no): new lines are inserted and lines of claims adjudicated later are updated in place. Finally, fact_claim_monthly_snapshot is rebuilt by grouping the fact table by submission month, insurer and hospital, indexes are created and an audit row is written.

Output: fact_claim_line, fact_claim_monthly_snapshot and the etl_audit record.

5.5 OLAP Analytics, Testing and Reporting

Purpose: to answer business questions, verify correctness and present results.

Input: the warehouse and, for comparison, a cleansed normalized (3NF) copy of the source. **Process:** six OLAP queries (monthly trend, denial rate by insurer and year, top diagnoses by paid amount, hospital by claim type, age band by gender, denial reasons by insurer type) are run on both models, compared for identical results and timed (median of seven runs after one warm-up). Thirteen automated test cases check reconciliation, integrity, rules, history and idempotency. The dashboard and charts are drawn with matplotlib.

Output: result sets, the test report, response-time comparison and the dashboard.

Interaction of modules. M1 delivers delta rows to M2; M2 passes only clean rows to M3 and M4 and writes rejects to the audit store; M3 supplies surrogate keys and history to M4; M4 produces the fact tables that M5 queries and tests. Each new batch restarts the cycle at M1 with the watermark advanced by M4 (Fig. 7).

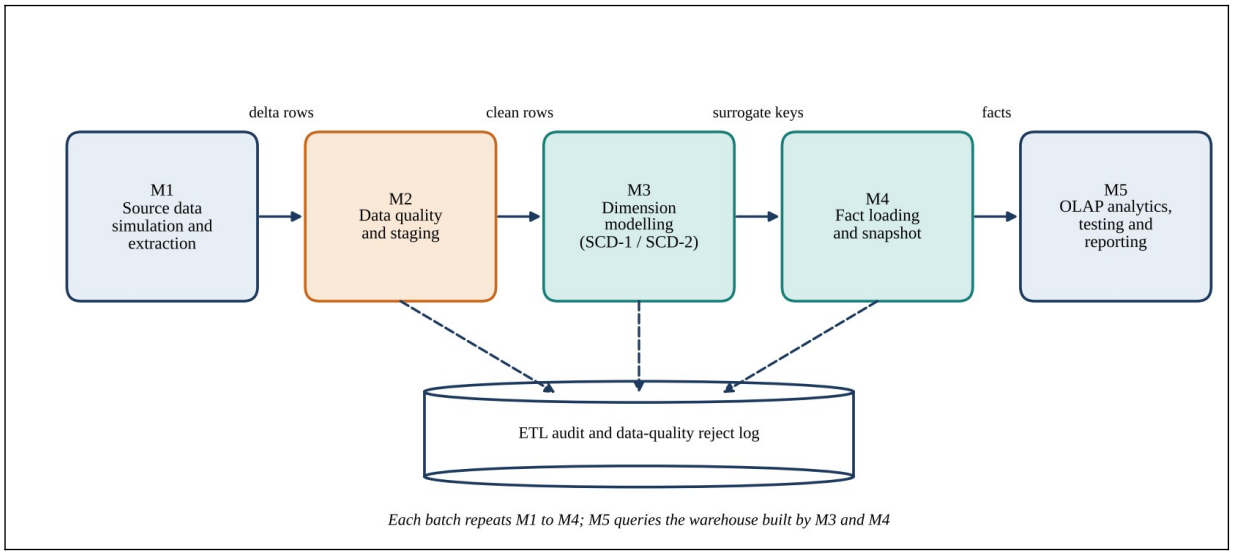


Fig. 7: Module Interaction Flow

CHAPTER - 6

SYSTEM IMPLEMENTATION

6.1 Source Data Simulation and Extraction Implementation

Steps followed: (1) define reference data; (2) generate patients with age, gender, city and plan; (3) draw claims per diagnosis with seasonality, hospital and insurer effects; (4) build itemised service lines; (5) assign final adjudication outcome and amounts; (6) inject defects; (7) publish batch 1 as of 31 December 2024 and batch 2 as of 31 December 2025; (8) extract with a watermark.

Algorithm: 1. Read the last watermark from etl_audit. 2. Select claims and lines whose last_updated is greater than the watermark. 3. Read the (small) reference tables and the patient master in full. 4. Set the new watermark to the maximum last_updated seen.

The probability of denial for a claim is 0.07 multiplied by an insurer factor (0.7 to 1.5), a diagnosis factor and 1.35 for reimbursement claims; partial approval has a fixed probability of 0.11; the adjudication delay is drawn from a gamma distribution whose mean depends on the insurer and on cashless or reimbursement type. Claims whose adjudication date is later than the batch date are exposed as Pending, which creates the late-arriving updates handled in module 5.4.

Key code (watermark extraction):

```
wm = dw.execute("SELECT COALESCE(MAX(watermark_out),'1900-01-01')
                 FROM etl_audit").fetchone()[0]
C = pd.read_sql("SELECT * FROM src_claim WHERE last_updated > ?",
                src, params=[wm])
L = pd.read_sql("SELECT * FROM src_claim_line WHERE last_updated > ?",
                src, params=[wm])
wm_out = max(C.last_updated.max(),          L.last_updated.max()) if len(C)
else wm
```

Parameters: seed 2026, 25,000 patients, 100,000 claims, 2023-01-01 to 2025-12-20 admissions, insurers' mean adjudication lag 9 to 30 days.

6.2 Data Quality and Staging Implementation

Steps followed: the delta claims are de-duplicated, the claim lines are tested against rules R2 to R5 in order (a line is counted under the first rule it violates), invalid diagnosis codes are mapped to the Unknown member, and every event is appended to dq_reject. Fig. 8 shows the control flow of one batch.

Pseudo-code: for each delta claim header: keep first occurrence of claim_id (R1). for each delta claim line: if billed < 0 then reject (R2); else if paid > allowed then reject (R3); else if

service date outside calendar then reject (R4); else if no parent claim then reject (R5); else accept. for each accepted claim: if ICD-10 not in reference then map to Unknown (R6).

Key code (rules):

```
dup  = C.duplicated('claim_id', keep='first')           # R1
C    = C[~dup].copy()
bad2 = L.billed_amount < 0                              # R2
bad3 = (~bad2) & (L.paid_amount > L.allowed_amount+0.005) # R3
sd   = pd.to_datetime(L.service_date)
bad4 = (~bad2) & (~bad3) & ((sd < '2023-01-01') |
                            (sd > '2025-12-31'))        # R4
bad5 = ~L.claim_id.isin(C.claim_id)                     # R5
L = L[~(bad2 | bad3 | bad4 | bad5)]       # rejected rows go to dq_reject
unk = ~C.primary_icd10.isin(valid_icd10)                # R6
C.loc[unk, 'primary_icd10'] = 'UNK'       # mapped to Unknown member -1
```

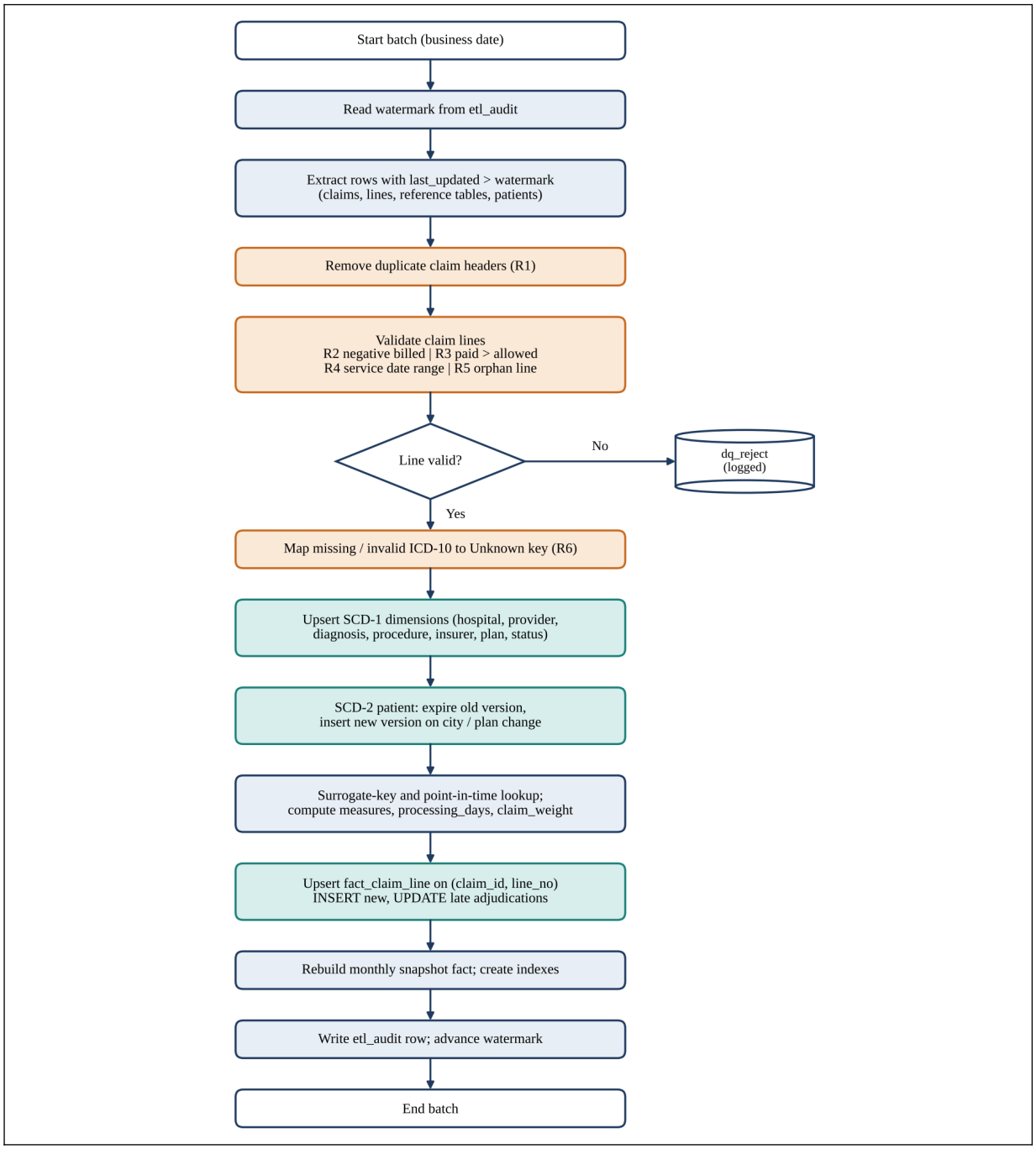


Fig. 8: ETL Batch Flowchart

The rule order matters for reporting only; rejected lines are never loaded. R6 does not reject the claim because its financial amounts are still valid and should appear in totals under an Unknown diagnosis.

6.3 Dimension Modelling and SCD Implementation

Steps followed: the calendar dimension (2022 to 2026) is generated with fiscal year (April to March) and weekend flag. Type-1 dimensions are merged on their natural key: the existing table is read, new keys are inserted and changed attributes updated. The junk dimension

receives new combinations of status, denial reason, claim type and admission type. For dim_patient the current version of each patient is compared with the incoming record.

Algorithm (Type-2): 1. Left-join incoming patients to current dimension rows. 2. No match: insert version 1 with effective_from 2022-01-01. 3. Match but city, state or plan differs: set effective_to = profile_updated_on − 1 day and is_current = 0 on the old row; insert version n + 1 effective from profile_updated_on. 4. Refresh age_band on all versions.

Key code (Type-2 change):

```
m = P.merge(current_rows, on='patient_id', how='left')
new = m[m.patient_key.isna()]                  # first time seen
chg = m[m.patient_key.notna() & ((m.city != m.city_cur) |
        (m.state != m.state_cur) | (m.plan_id != m.plan_id_cur))]
for r in chg.itertuples():                     # close the old version
    eff = date.fromisoformat(r.profile_updated_on)
    con.execute('UPDATE dim_patient SET effective_to=?, is_current=0'
                ' WHERE patient_key=?', (eff - timedelta(days=1),
r.patient_key))
# open version n+1 with effective_from = profile_updated_on,
# effective_to = '9999-12-31', is_current = 1
```

In batch 1 the load created 25,000 patient versions; in batch 2 it added 2,391 new versions for patients whose plan or city changed on 1 January 2025. All other dimensions were loaded with no changed attributes in batch 2, as expected.

6.4 Fact Loading, Snapshot and Final Integration Implementation

Steps followed: claim lines are joined to claims, plans and every dimension to obtain surrogate keys; the patient version valid on the admission date is selected; date keys, processing_days and claim_weight are computed; rows already present are counted as updates and the rest as inserts; all rows are written with an upsert; indexes are created on all foreign keys; the monthly snapshot is rebuilt; and an audit row is written. The unit of work is one batch and the ETL command is run twice, python etl.py 1 and python etl.py 2.

Pseudo-code: for each valid line: look up eight dimension keys; choose patient version by admission date; compute processing_days = adjudication − submission; compute claim_weight = 1 / lines in claim; upsert on (claim_id, line_no). then rebuild snapshot and write audit.

Key code (fact load and snapshot):

```
F = F[(F.admission_date >= F.effective_from) &
      (F.admission_date <= F.effective_to)]      # point-in-time lookup
F['processing_days'] = (pd.to_datetime(F.adjudication_date)
                        - pd.to_datetime(F.submit_date)).dt.days
F['claim_weight'] = 1.0 / F.groupby('claim_id').claim_id.transform('size')
con.executemany("INSERT INTO fact_claim_line(...) VALUES(...) "
  "ON CONFLICT(claim_id,line_no) DO UPDATE SET " + update_list, rows)
```

```
con.execute("INSERT INTO fact_claim_monthly_snapshot "
  "SELECT d.year*100+d.month, f.insurer_key, f.hospital_key, "
  "SUM(f.claim_weight), COUNT(*), SUM(f.billed_amount), ... "
  "FROM fact_claim_line f JOIN dim_date d ON d.date_key=f.submit_date_key "
  "GROUP BY 1,2,3")
```

Load profile: batch 1 (initial) extracted 359,423 lines, rejected 1,553 and inserted 357,870 in 14 s. Batch 2 (incremental) extracted 219,716 lines, rejected 949, inserted 210,773 and updated 7,994 late-adjudicated lines in 13 s. The final fact table holds 568,643 lines. Fig. 9 summarises the rows processed and the rejects by rule. No machine-learning model is used in this project, so there are no training epochs or learning rates; the evaluation metrics are reconciliation, test outcomes and response time (Chapter 7).

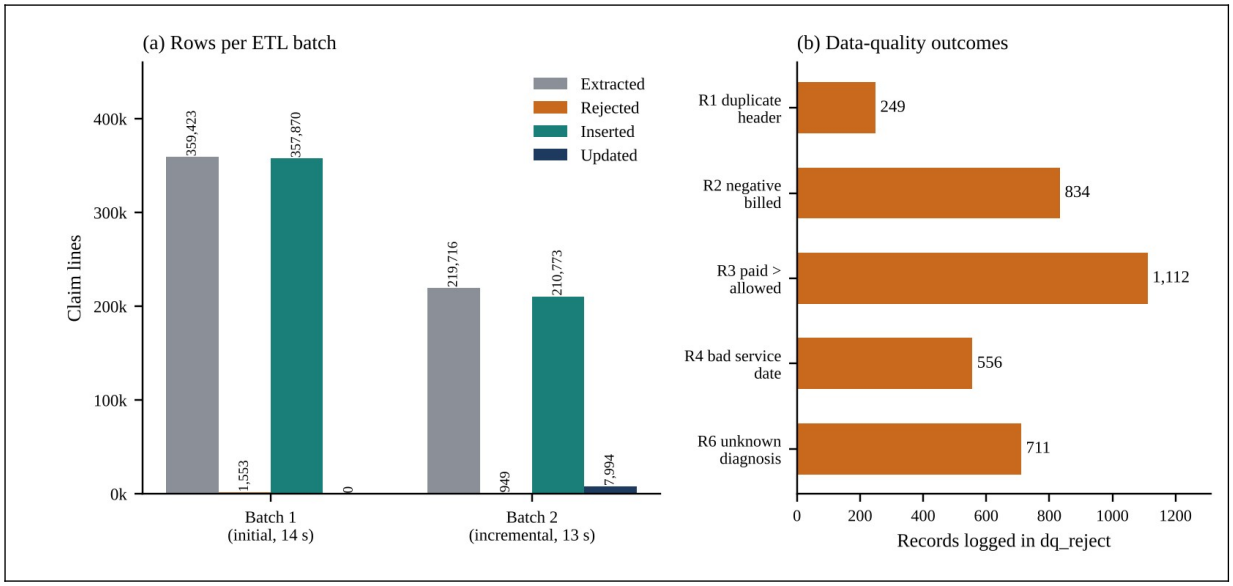


Fig. 9: ETL Load Profile and Data-Quality Outcomes

CHAPTER - 7

TESTING AND OUTPUT SCREENS

7.1 Test Cases and Test Results

Thirteen automated test cases were executed after both batches (tests.py). They cover valid input, invalid input (injected defects), boundary behaviour (calendar and unknown members) and repeated execution. All test cases passed.

TC ID	Test Description	Input / Check	Expected Output	Actual Output	Status
TC01	Row-count reconciliation	Source lines passing DQ rules vs fact rows	568,643	568,643	Pass
TC02	Paid-amount reconciliation	SUM(paid) source vs fact	3,483,995,473.32	3,483,995,473.32	Pass
TC03	Referential integrity	Orphan foreign keys across 11 fact-to-dimension links	0	0	Pass
TC04	Duplicate claim headers removed	249 duplicate headers injected; distinct claims compared	100,000	100,000	Pass
TC05	Negative billed amount rejected	834 negative lines injected	834 rejected, 0 in fact	834 rejected, 0 in fact	Pass
TC06	Missing / invalid diagnosis mapped to Unknown	NULL and invalid ICD-10 codes injected	698 claims on key -1	698 claims on key -1	Pass
TC07	SCD Type-2 versioning	2391 patients changed city or plan	25,000 current rows; 2391 new versions; 0 overlaps	25,000 current rows; 2391 new versions; 0 overlaps	Pass
TC08	Point-in-time dimension lookup	Admission date must fall in the linked patient version's validity period	0	0	Pass
TC09	Date dimension continuity	Calendar 2022-01-01 to 2026-12-31 without gaps	1,826 days	1,826 rows	Pass
TC10	Snapshot fact reconciliation	Monthly snapshot vs transaction fact (paid amount, claim count)	3,483,995,473.32; 100,000	3,483,995,473.32; 100,000	Pass
TC11	Query-result equivalence	Six OLAP queries run on 3NF copy and on star schema	6 of 6 identical (+ snapshot)	6 of 6 identical	Pass
TC12	Late-arriving adjudication update	1,413 claims pending in batch 1 adjudicated later	0 stale Pending claims; lines updated in place	0 stale; 7,994 lines updated	Pass
TC13	Idempotent re-load (replay of batch 2)	Watermark reset; batch 2 data loaded again	0 new rows; totals unchanged (568,643)	0 new rows; totals unchanged (568,643)	Pass

7.2 Output Screen 1 – ETL Execution Console

```
$ python etl.py 1 && python etl.py 2

[06:54:08] ===== ETL batch 1  (business date 2024-12-31) =====
[06:54:10] EXTRACT   watermark 1900-01-01 -> 2024-12-31: 63,188 claim rows, 359,423 line rows, 25,000 patients
           (2.2s)
[06:54:10] TRANSFORM data-quality gate: 157 duplicate headers removed, 1553 lines rejected (R2=514, R3=702,
           R4=337, R5=0), 428 claims mapped to Unknown diagnosis  (0.3s)
[06:54:11] DIMENSIONS dim_date +1827, hospital/provider/diagnosis/procedure/insurer/plan (SCD-1) new=219
           changed=0, dim_patient (SCD-2) +25000 versions, dim_claim_status +72  (0.4s)
[06:54:21] FACT      fact_claim_line: 357,870 inserted, 0 updated (late-arriving adjudication)  (9.7s)
[06:54:22] SNAPSHOT  fact_claim_monthly_snapshot rebuilt: 1,152 rows; indexes created  (1.5s)
[06:54:22] BATCH 1 COMPLETE in 14.1s  (rows in fact: 357,870)
[06:54:23] ===== ETL batch 2  (business date 2025-12-31) =====
[06:54:24] EXTRACT   watermark 2024-12-31 -> 2025-12-31: 38,474 claim rows, 219,716 line rows, 25,000 patients
           (1.1s)
[06:54:24] TRANSFORM data-quality gate: 92 duplicate headers removed, 949 lines rejected (R2=320, R3=410,
           R4=219, R5=0), 283 claims mapped to Unknown diagnosis  (0.2s)
[06:54:24] DIMENSIONS dim_date +0, hospital/provider/diagnosis/procedure/insurer/plan (SCD-1) new=0 changed=0,
           dim_patient (SCD-2) +2391 versions, dim_claim_status +0  (0.4s)
[06:54:34] FACT      fact_claim_line: 210,773 inserted, 7,994 updated (late-arriving adjudication)  (10.2s)
[06:54:35] SNAPSHOT  fact_claim_monthly_snapshot rebuilt: 1,728 rows; indexes created  (0.7s)
[06:54:35] BATCH 2 COMPLETE in 12.5s  (rows in fact: 568,643)
```

Fig. 10: ETL Execution Console (Output Screen 1)

Fig. 10 shows the console output of the two ETL runs. For each batch the log shows the extract step with the watermark and row counts, the data-quality gate with the number of duplicate headers and rejected lines by rule, the dimension step with new and changed members, the fact step with inserted and updated lines, the snapshot rebuild and the batch summary with duration. The screen is reached by running python etl.py 1 followed by python etl.py 2 in the project folder.

7.3 Output Screen 2 – Claims Analytics Dashboard

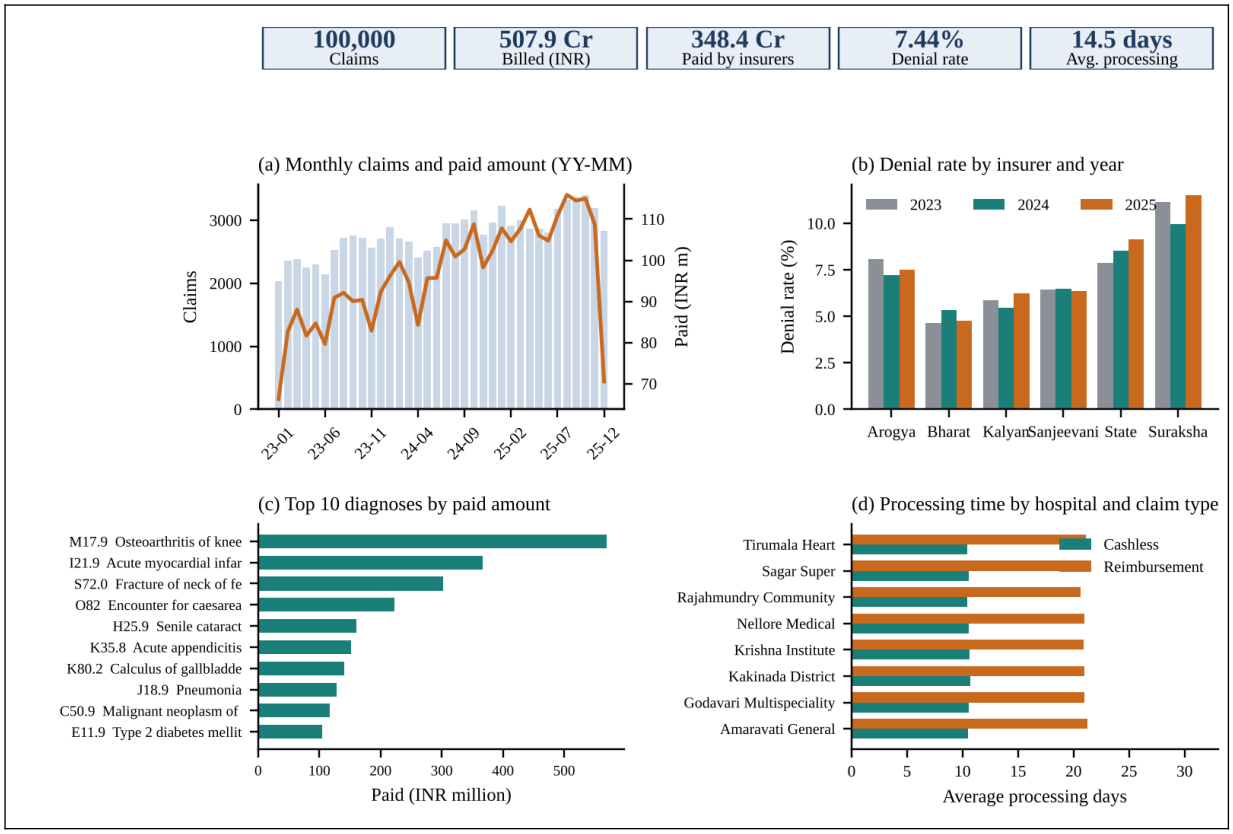


Fig. 11: Claims Analytics Dashboard (Output Screen 2)

Fig. 11 is the claims dashboard produced from warehouse queries. The key figures are 100,000 claims, a billed amount of 507.9 crore rupees, 348.4 crore paid by insurers, an overall denial rate of 7.44% of adjudicated claims and an average processing time of 14.5 days. Panel (a) shows monthly claims and paid amount (the last month is low because many claims are still pending), panel (b) shows the denial rate by insurer and year, panel (c) lists the top ten diagnoses by paid amount, and panel (d) compares processing time for cashless and reimbursement claims by hospital.

The Type-2 patient dimension changes the answer to some questions. For the patients whose city changed on 1 January 2025, the table below counts their claims by the city recorded at the time of care and by their current city; the normalized source keeps only the current city and therefore cannot reproduce the first column.

City	Claims by city at time of care (Type-2)	Claims by current city (Type-1 view)	Difference
Vijayawada	1,435	1,325	+110
Eluru	1,416	1,275	+141
Visakhapatnam	1,335	1,183	+152
Guntur	1,094	1,131	-37
Hyderabad	1,041	1,059	-18
Rajahmundry	895	990	-95

7.4 Performance Results and Discussion

Each of the six queries was run seven times after a warm-up on the cleansed normalized copy (100,000 claims and 568,643 lines, indexed on all keys) and on the star schema (568,643 fact rows); the table lists the median. The results of all six queries were identical in both models, which confirms that the warehouse preserves the business meaning of the source.

Metric / Method	Existing Method (normalized 3NF copy)	Proposed Method (star schema)
Q1: Monthly claims, billed and paid amount (roll-up to month)	865 ms	233 ms (3.7x faster)
Q2: Denial rate by insurer and year (slice and dice)	177 ms	439 ms (2.5x slower)
Q3: Top 10 diagnoses by insurer-paid amount	502 ms	96 ms (5.2x faster)
Q4: Hospital by claim type: claims, avg processing days, settlement ratio	318 ms	589 ms (1.9x slower)
Q5: Utilisation by age band and gender	1043 ms	419 ms (2.5x faster)
Q6: Denied amount by denial reason and insurer type	228 ms	144 ms (1.6x faster)
Q1 on monthly snapshot fact	865 ms (no equivalent)	0.3 ms (2,884x faster)
Total of Q1 to Q6	3133 ms	1919 ms (1.63x faster)
Identical results (6 of 6 queries)	Reference	Yes
Database size on disk	84.7 MB	142.1 MB (includes indexes, snapshot and logs)

The star schema was faster on 4 of the 6 queries: Q1 (3.7x), Q3 (5.2x), Q5 (2.5x) and Q6 (1.6x), because it needs fewer joins and uses precomputed date attributes instead of date functions on text. It was slower on Q2 and Q4 (about 2.5x and 1.9x), which are claim-level questions: the normalized copy answers them from the 100,000-row claim header table, whereas the star schema scans the 568,643-row line-grain fact table and must weight lines. This is an honest limitation of a single line-grain fact table. It can be removed by a claim-grain fact or aggregate, which is what the monthly snapshot does: Q1 answered from the snapshot took 0.3 ms, about 2,884 times faster than the normalized copy. Over all six queries the total time fell from 3133 ms to 1919 ms. The warehouse database is larger (142.1 MB against 84.7 MB) because it stores keys, indexes, the snapshot and the audit tables.

The data-quality results are consistent with the injected defects: all 834 negative lines, 1,112 lines with paid above allowed and 556 invalid-date lines were rejected, all 249 duplicate headers were removed and all 698 claims with missing or invalid diagnosis were assigned to the Unknown member (test cases TC04 to TC06).

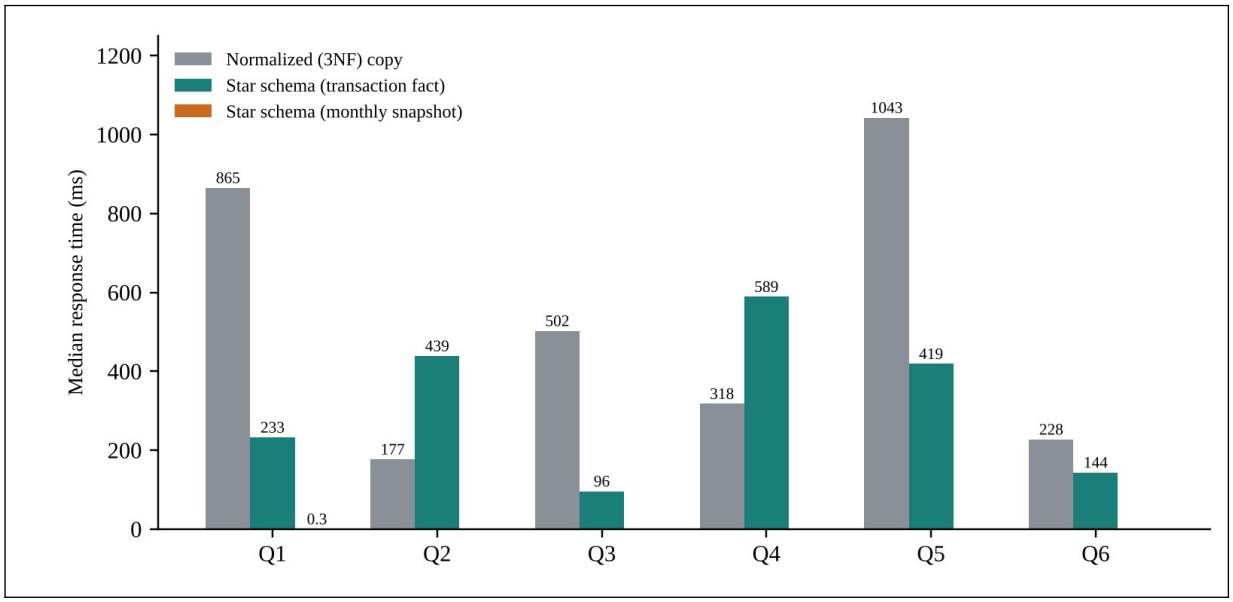


Fig. 12: Query Response-Time Comparison (median of seven runs)

CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

This project designed and built a fact-dimension data warehouse for hospital insurance claims. The model consists of a claim-line transaction fact table, a monthly snapshot fact table and nine conformed dimensions, with a Type-2 patient dimension and an Unknown diagnosis member. An incremental ETL pipeline in Python and SQLite loaded a synthetic source system of 100,000 claims and 571,145 claim lines in two batches, rejecting 2,502 invalid lines, removing 249 duplicate headers and storing 568,643 lines in the fact table.

All 13 automated test cases passed, including reconciliation of row counts and paid amounts, referential integrity, point-in-time history, late-arriving updates and idempotent replay. The six OLAP queries returned identical results on the star schema and on a normalized copy. The star schema was faster on four queries (up to 5.2 times) and slower on two claim-level queries; the snapshot fact answered the monthly trend 2,884 times faster than the normalized copy. These measurements describe one machine and synthetic data and should be repeated on real data before conclusions are generalised.

The work showed that a small, carefully modelled warehouse can give hospital finance teams consistent denial rates, settlement ratios and processing times, and can explain why history tracking and data-quality logging matter: for the relocated patients, claims by city differed between the history-correct and the current-state view. Practical lessons were the value of declaring the grain first, of weighting lines to count claims without COUNT(DISTINCT), and of testing the warehouse against an independent reference.

The warehouse is a base for further analysis such as denial prediction, fraud scoring, payer scorecards and revenue-cycle monitoring, because these tasks need the clean, integrated and historical claims data that it provides.

8.2 Future Enhancements

- Replace the synthetic source by de-identified real hospital billing data and validate the rules and metrics with the finance department.
- Add a claim-grain fact table or aggregate tables to speed up claim-level questions such as Q2 and Q4.
- Move the warehouse to a server database (PostgreSQL or MySQL) with partitioning, and schedule the ETL with a workflow tool for daily loads.

- Add further fact tables, for example pre-authorisation requests and payment transactions, sharing the same conformed dimensions, to form a fact constellation.
- Build an interactive web dashboard with role-based access, and apply masking and encryption for personal data.
- Use the warehouse as input for machine-learning models for denial prediction and fraud detection, and map the model to OMOP for interoperability [7], [8].

REFERENCES

The references are numbered in the order of first citation in the text.

[1] R. Kimball and M. Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Indianapolis, IN, USA: Wiley, 2013.

[2] W. H. Inmon, Building the Data Warehouse, 4th ed. Indianapolis, IN, USA: Wiley, 2005.

[3] S. Chaudhuri and U. Dayal, “An overview of data warehousing and OLAP technology,” ACM SIGMOD Record, vol. 26, no. 1, pp. 65–74, 1997.

[4] Z. Wang et al., “Clinical data warehousing: A scoping review,” J. Soc. Clin. Data Manag., vol. 4, no. 2, 2024, doi: 10.47912/jscdm.320.

[5] S. Lyu, S. Craig, G. O’Reilly and D. Taniar, “The development and use of data warehousing in clinical settings: a scoping review,” Front. Digit. Health, vol. 7, Art. no. 1599514, 2025, doi: 10.3389/fdgth.2025.1599514.

[6] G. Turcan and S. Peker, “A multidimensional data warehouse design to combat the health pandemics,” J. Data Inf. Manag., vol. 4, no. 3–4, pp. 371–386, 2022, doi: 10.1007/s42488-022-00082-6.

[7] J.-W. Kim et al., “Scalable infrastructure supporting reproducible nationwide healthcare data analysis toward FAIR stewardship,” Sci. Data, vol. 10, Art. no. 674, 2023, doi: 10.1038/s41597-023-02580-7.

[8] M. Finster, M. Moinat and E. Taghizadeh, “ETL: From the German Health Data Lab data formats to the OMOP Common Data Model,” PLOS ONE, vol. 20, no. 1, pp. 1–21, 2025, doi: 10.1371/journal.pone.0311511.

[9] O. Cherkaoui, H. Anoun and A. Maizate, “A benchmark of health insurance fraud detection using machine learning techniques,” IAES Int. J. Artif. Intell., vol. 13, no. 2, pp. 1925–1934, 2024, doi: 10.11591/ijai.v13.i2.pp1925-1934.

[10] C. Rinner, D. Gezgin, C. Wendl and W. Gall, “A clinical data warehouse based on OMOP and i2b2 for Austrian health claims data,” Medical University of Vienna, 2018. [Online]. Available: https://www.meduniwien.ac.at/msi/mias/papers/Rinner2018b.pdf

[11] W. Su, “Study of key technologies of insurance claims based on data mining,” in Proc. 3rd Int. Conf. Materials Engineering, Manufacturing Technology and Control (ICMEMTC), Atlantis Press, 2016.

[12] SQLite Consortium, “SQLite documentation.” [Online]. Available: https://www.sqlite.org/docs.html (Accessed: 19 Oct. 2026).

[13] W. McKinney, “Data structures for statistical computing in Python,” in Proc. 9th Python in Science Conf. (SciPy), 2010, pp. 56–61.

[14] J. D. Hunter, “Matplotlib: A 2D graphics environment,” Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, 2007.

[15] E. F. Codd, S. B. Codd and C. T. Salley, “Providing OLAP to user-analysts: An IT mandate,” Codd & Associates, 1993.

APPENDIX

Appendix A: Source Code

```
# etl.py - SCD Type-2 maintenance of dim_patient (excerpt)
def scd2_patient(con, P, as_of):
    cur = pd.read_sql('SELECT * FROM dim_patient WHERE is_current=1', con)
    P['age_band'] = age_band(P.birth_date, as_of)
    m = P.merge(cur[['patient_key','patient_id','city','state',
                     'plan_id','version_no']],
                on='patient_id', how='left', suffixes=('', '_cur'))
    new = m[m.patient_key.isna()]
    chg = m[m.patient_key.notna() & ((m.city != m.city_cur) |
            (m.state != m.state_cur) | (m.plan_id != m.plan_id_cur))]
    for r in chg.itertuples():
        eff = dt.date.fromisoformat(r.profile_updated_on)
        con.execute('UPDATE dim_patient SET effective_to=?, is_current=0'
                    ' WHERE patient_key=?',
                    ((eff - dt.timedelta(days=1)).isoformat(),
                     int(r.patient_key)))
    # insert version 1 for new patients and version n+1 for changed ones
    ...

# etl.py - data-quality gate (excerpt of run_batch)
dup  = C.duplicated('claim_id', keep='first'); C = C[~dup].copy()
bad2 = L.billed_amount < 0
bad3 = (~bad2) & (L.paid_amount > L.allowed_amount + 0.005)
sd   = pd.to_datetime(L.service_date)
bad4 = (~bad2) & (~bad3) & ((sd < '2023-01-01') | (sd > '2025-12-31'))
bad5 = (~bad2) & (~bad3) & (~bad4) & (~L.claim_id.isin(C.claim_id))
L = L[~(bad2 | bad3 | bad4 | bad5)].copy()
unk = ~C.primary_icd10.isin(valid_icd10); C.loc[unk,'primary_icd10']='UNK'

# etl.py - fact load by upsert
F = F[(F.admission_date >= F.effective_from) &
      (F.admission_date <= F.effective_to)]
F['processing_days'] = (pd.to_datetime(F.adjudication_date)
                        - pd.to_datetime(F.submit_date)).dt.days
F['claim_weight'] = 1.0 / F.groupby('claim_id').claim_id.transform('size')
dw.executemany(
  f'INSERT INTO fact_claim_line({cols}) VALUES({marks}) '
  f'ON CONFLICT(claim_id,line_no) DO UPDATE SET {update_list}', rows)

-- olap.py - Q2 denial rate by insurer and year (star schema)
SELECT i.insurer_name, d.year, ROUND(SUM(f.claim_weight)) AS claims,
  ROUND(SUM(CASE WHEN s.claim_status='Denied' THEN f.claim_weight END)) AS denied,
  ROUND(100.0*SUM(CASE WHEN s.claim_status='Denied' THEN f.claim_weight END)
   /SUM(CASE WHEN s.claim_status<>'Pending' THEN f.claim_weight END),2)
FROM fact_claim_line f JOIN dim_insurer i ON i.insurer_key=f.insurer_key
JOIN dim_date d ON d.date_key=f.submit_date_key
JOIN dim_claim_status s ON s.claim_status_key=f.claim_status_key
GROUP BY 1,2 ORDER BY 1,2;
```

The complete source code (generate_source.py, etl.py, olap.py, tests.py and the figure scripts) is supplied with the project files.

Appendix B: Plagiarism Report and Achievements

Plagiarism / similarity report: Similarity index obtained from the department's plagiarism check: _____ %. The report sheet is attached after this page.

Achievements: Nil.